

# **ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY**

A UNIT OF ALVA'S EDUCATION FOUNDATION, MOOBBIDRI

**MANGALORE-574225**



**A PROJECT REPORT  
ON**

## **AGRILINK-OR: DETERMINISTIC DECISION-SUPPORT & OPERATIONS RESEARCH SYSTEM FOR AGRICULTURAL SUPPLY CHAINS**

**SUBMITTED BY:**

**NANDISHWAR EN  
DIVYA VS**

**4AL23IC026  
4AL23CG011**

**UNDER THE GUIDANCE OF:**

**ANJIT RAJA R  
(TRAINER BE PRACTICAL)**

## Table of Contents

---

|                                                             |    |
|-------------------------------------------------------------|----|
| 1. Abstract.....                                            | 3  |
| 2. Introduction and Problem Statement.....                  | 4  |
| 3. Data.....                                                | 6  |
| 4. System Architecture.....                                 | 8  |
| 5. Data Model .....                                         | 10 |
| 6. Analytical Modules .....                                 | 12 |
| 7. API Service.....                                         | 14 |
| 8. User Interface.....                                      | 15 |
| 9. Worked Example .....                                     | 17 |
| 10. Testing and Quality Assurance.....                      | 19 |
| 11. Design Rationale: Why There Is No Machine Learning..... | 20 |
| 12. Limitations and Integrity .....                         | 20 |
| 13. Roadmap.....                                            | 21 |
| Appendix A. Commands and Runtime .....                      | 22 |
| Appendix B. Technology Choices .....                        | 22 |
| Appendix C. Assumption Parameters .....                     | 22 |
| Appendix D. Repository Map .....                            | 23 |
| Appendix E. Glossary .....                                  | 23 |
| Appendix F. Verification Notes .....                        | 24 |

**About this report** Every figure and formula here comes from the project's ground-truth summary or from arithmetic on it. Items that could not be verified are marked where they occur and collected in Appendix F.

## 1. Abstract

---

**AgriLink-OR** is a deterministic decision-support system designed to help Farmer Producer Organisations (FPOs) make informed decisions about agricultural commodity marketing. The system addresses three practical questions: **where to sell, whether to hold, and how to aggregate produce for bulk orders**. Unlike machine-learning-based forecasting systems, AgriLink-OR uses transparent mathematical and optimisation techniques so that every recommendation can be inspected, reproduced, and explained.

The system operates on the Karnataka subset of an **Agmarknet agricultural price dataset**, containing 14,352 cleaned records from **72 mandis across 21 districts**, covering five crops—Onion, Potato, Tomato, Wheat, and Rice—from June 2023 to June 2025. The data-processing pipeline identifies and handles issues such as inconsistent date formats, unit errors, missing observations, and incorrect crop assumptions. Mandi locations are geocoded and farm-to-mandi distances are calculated to incorporate transportation costs into decision-making.

The analytical framework consists of four major modules. **M1** applies Seasonal-Trend decomposition using Loess (STL) to identify seasonal patterns, trends, volatility bands, and potential GLUT/SPIKE periods. **M2** calculates net-in-hand realisation by considering mandi price, freight, cess, commission, handling, and transit losses, thereby ranking mandis based on actual expected returns rather than board prices alone. **M3** evaluates hold-versus-sell decisions using break-even analysis and a **Mixed-Integer Linear Programming (MILP)** model that incorporates storage costs, financing costs, shrinkage, and multiple selling tranches. **M4** uses a bounded-knapsack optimisation model to identify farmer lots that can satisfy bulk orders while minimising surplus.

The system is implemented using **Python, Pandas, Parquet, PostgreSQL, SQLAlchemy, FastAPI, Streamlit, Plotly, Folium, PuLP, and CBC**. Its architecture separates pure analytical modules from the database and application layers, enabling reproducible computation and testing. The platform provides a dashboard and API for decision analysis while explicitly reporting data limitations and refusing to generate unsupported predictions.

AgriLink-OR demonstrates how deterministic analytics, optimisation, and transparent data engineering can be combined to support practical agricultural marketing decisions while maintaining auditability and reproducibility.

## 2. Introduction and Problem Statement

### 2.1 Context

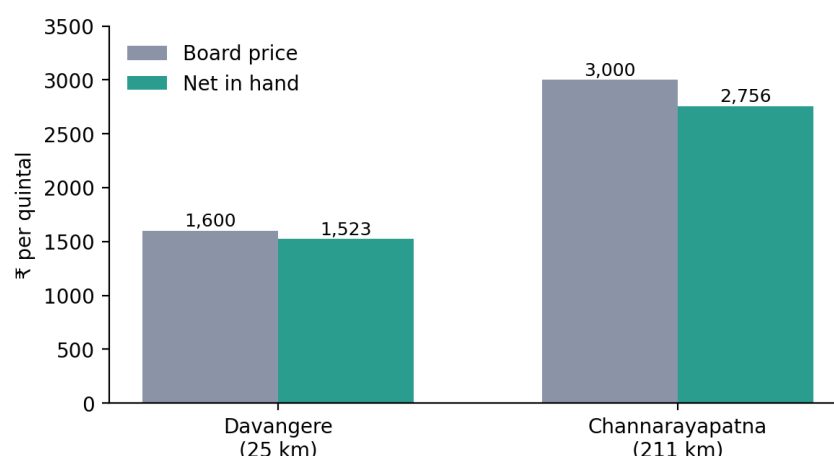
Smallholder farmers rarely have the market information, transport scale or storage access that large traders have. FPOs exist to close that gap by pooling produce, but a pooled lot only helps if it is sold well. The FPO's manager must choose a market, a day and a lot composition, usually with a price board from a single nearby mandi and personal experience as the only guide.

Public price data exists: the Agmarknet system records daily arrivals-market prices for hundreds of mandis. What has been missing is a tool that turns those records into a defensible recommendation that a committee can question. That is the gap AgriLink-OR addresses.

### 2.2 The Obvious Answer Fails

The obvious rule is to sell where the board price is highest. It fails because the board price is not what the FPO receives. Freight, market cess, commission, loading and unloading, and shrinkage in transit all come out of the sale. A mandi with a high board price and a long haul can pay less than a nearer market.

The worked example in Chapter 9 shows the size of the effect. The nearest mandi, Davangere at 25 km, quotes ₹1,600 per quintal and nets ₹1,523. The best market, 211 km away, quotes ₹3,000 and nets ₹2,756. Ranking by distance would have cost the FPO roughly ₹1,233 per quintal. Ranking by board price alone would have ignored ₹244 per quintal of costs.



*Figure 2.1. Board price versus net in-hand realisation at the nearest and the best mandi (Onion, 200 qtl).*

## 2.3 The Costs an FPO Bears

| Cost                       | What drives it                                                     | Modelled in |
|----------------------------|--------------------------------------------------------------------|-------------|
| Freight                    | Trucks required, diesel price, mileage, distance, empty return leg | M2          |
| Cess and commission        | Percentage of gross sale value                                     | M2          |
| Handling                   | Loading and unloading per quintal, plus a fixed charge             | M2          |
| Transit loss               | Shrinkage between depot and market                                 | M2          |
| Storage rent and insurance | Days held                                                          | M3          |
| Financing cost             | Price level, loan-to-value ratio, loan rate, opportunity rate      | M3          |
| Storage shrinkage          | Per-day loss while held                                            | M3          |

## 2.4 Decisions Are Made as of a Date

Every computation takes an explicit **as\_of** date and uses only rows on or before it. This point-in-time discipline has two benefits. Any past decision can be reproduced exactly, and the system can in principle be replayed against what the market did next. It also prevents the quiet look-ahead error that plagues price analysis, in which future rows leak into a supposedly historical calculation.

## 2.5 Objectives

- Produce three actionable answers (sell, hold, aggregate) from real public data.
- Make every headline number traceable to a formula, a parameter, or a data row.
- Keep the analytical core free of machine learning so that every step can be inspected and argued with.
- Separate pure mathematics from data access so that the system can be tested and its data source swapped.
- Refuse to fabricate: state a reason in place of an invented figure when the data cannot support one.

## 3. Data

### 3.1 Source and Provenance

The input is [Agriculture\\_price\\_dataset.csv](#), a public Agmarknet price dump covering many states. The project works on its Karnataka slice.

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| Source file       | <a href="#">Agriculture_price_dataset.csv</a> (public Agmarknet price dump) |
| Source volume     | 737,392 rows across 30 state labels                                         |
| Date span         | 6 June 2023 to 11 June 2025                                                 |
| Karnataka extract | 14,352 rows · 72 mandis · 21 districts · 5 crops                            |
| Arrivals column   | Absent                                                                      |
| Fallback policy   | None. Missing data yields a stated reason, not an invented price            |

**Limitation** With no arrivals column the GLUT and SPIKE flags are residual z-score proxies for supply pressure. They are indicators inferred from prices and are never measurements of arrivals.

### 3.2 The Karnataka Extract

Five crops are present. Onion and Potato carry enough history at enough mandis to support a 52-week seasonal model. Tomato, Wheat and Rice do not, and the system reports that plainly for them.

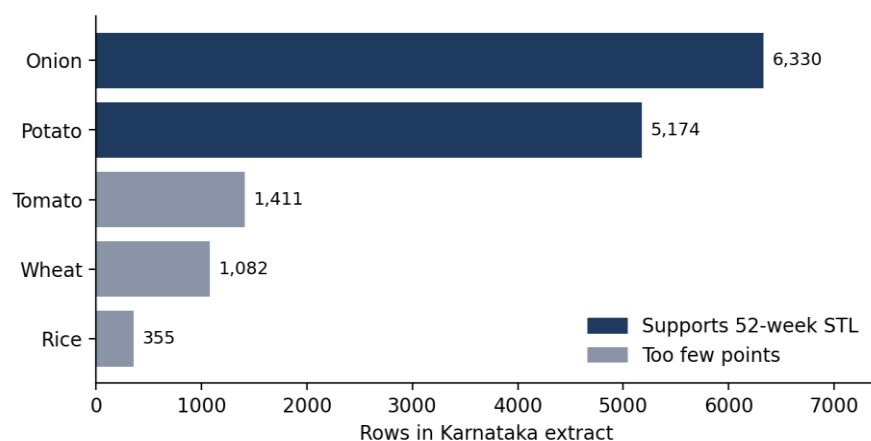


Figure 3.1. Rows per crop in the Karnataka extract; Onion and Potato support the annual model.

| Crop   | Rows  | Share | Supports 52-week STL |
|--------|-------|-------|----------------------|
| Onion  | 6,330 | 44.1% | Yes                  |
| Potato | 5,174 | 36.1% | Yes                  |

| Crop         | Rows          | Share       | Supports 52-week STL |
|--------------|---------------|-------------|----------------------|
| Tomato       | 1,411         | 9.8%        | No                   |
| Wheat        | 1,082         | 7.5%        | No                   |
| Rice         | 355           | 2.5%        | No                   |
| <b>Total</b> | <b>14,352</b> | <b>100%</b> |                      |

### 3.3 Geocoding Provenance

Distances need coordinates, and the dataset has none. Mandi locations were resolved through Nominatim and cached to CSV so that the pipeline is reproducible offline. Where the market itself could not be located, the district centroid was used and marked as such.

| mandis.source     | Mandis    | Meaning                              |
|-------------------|-----------|--------------------------------------|
| market            | 59        | Nominatim resolved the market itself |
| district_fallback | 13        | Pinned to the district centroid      |
| <b>Total</b>      | <b>72</b> |                                      |

The provenance is stored in `mandis.source`, exposed through `/api/meta`, and marked in the dashboard, so an approximate distance is never presented as an exact one.

### 3.4 The Five Data Traps

The dataset broke five assumptions that a reasonable engineer would have made. Each one corrupted the output in a specific way and each is now locked in by a test. They are documented here in full because they are the most transferable lessons in the project.

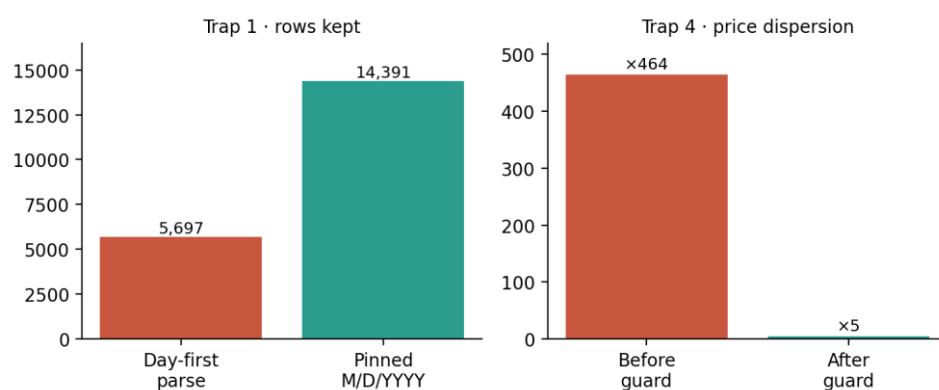


Figure 3.2. Left: rows retained under the two date parses. Right: price dispersion before and after the unit-error guard.

#### 3.4.1 Trap 1: An Ambiguous Date Format

|                          |                                           |
|--------------------------|-------------------------------------------|
| <b>What the data did</b> | Price Date is US M/D/YYYY, not day-first. |
|--------------------------|-------------------------------------------|

|                        |                                                                                                                  |
|------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Impact</b>          | Parsing with <code>dayfirst=True</code> silently discarded <b>8,694 of 14,391</b> Karnataka rows.                |
| <b>Fix</b>             | Pin the format explicitly ( <code>src/data/load_history.py:44</code> ), with a <code>dd/mm/yyyy</code> fallback. |
| <b>Regression test</b> | Pipeline test on parsed row count. Test name not available.                                                      |

### 3.4.2 Trap 2: The Default Crop Did Not Exist

|                          |                                                                  |
|--------------------------|------------------------------------------------------------------|
| <b>What the data did</b> | The default crop, Maize, does not appear in the Karnataka slice. |
| <b>Impact</b>            | A default request returned nothing.                              |
| <b>Fix</b>               | The default crop is now Onion, the best-supported crop.          |
| <b>Regression test</b>   | Default-run test. Test name not available.                       |

### 3.4.3 Trap 3: Interpolation Limits Destroyed the Sample

|                          |                                                                                                                                                |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What the data did</b> | Mandis report in bursts and leave holes of about four weeks.                                                                                   |
| <b>Impact</b>            | <code>interpolate(limit=4)</code> left every series below the <b>104 points</b> that a 52-week STL mathematically requires (two full periods). |
| <b>Fix</b>               | Fill the complete weekly grid and <b>report the imputed share</b> , so an interpolated series is never mistaken for an observed one.           |
| <b>Effect</b>            | Onion went from 0 to <b>12 usable mandis</b> .                                                                                                 |
| <b>Regression test</b>   | Per-mandi eligibility test. Test name not available.                                                                                           |

### 3.4.4 Trap 4: Rupees per Kilogram Recorded as Rupees per Quintal

|                          |                                                                                                                                                                                |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What the data did</b> | One mandi quoted prices per kilogram. Shimoga onion appeared at ₹14 against a ₹2,000 median.                                                                                   |
| <b>Impact</b>            | Price dispersion spanned <b>×464</b> .                                                                                                                                         |
| <b>Fix</b>               | A <b>one-sided</b> guard drops any quote below 10% of its own mandi-commodity median ( <code>UNIT_ERROR_FLOOR</code> , <code>src/data/clean.py</code> ). 39 rows were removed. |
| <b>Effect</b>            | Dispersion falls from <b>×464</b> to <b>×5.0</b> . GLUT and SPIKE counts are unchanged at 3 and 0.                                                                             |
| <b>Regression test</b>   | Cleaning test. Test name not available.                                                                                                                                        |

**Limitation** Only the low side is guarded, because Karnataka tomato genuinely trades at ₹13,000 per quintal. The guard could therefore mask a real price crash of more than 90%.

### 3.4.5 Trap 5: The Application Never Filtered by Crop

|                          |                                                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>What the code did</b> | <code>weekly_series(df[df.market == ref_mandi])</code> mixed Onion, Potato and Tomato modal prices into one series. |
| <b>Impact</b>            | The reference price was a blend of unrelated crops.                                                                 |
| <b>Fix</b>               | Filter by crop in the application, and enforce the same rule again at the API boundary.                             |
| <b>Regression test</b>   | <code>tests/test_app.py</code> (see Chapter 10).                                                                    |

## 4. System Architecture

### 4.1 Three Layers

The repository has three layers. Analytical modules are at the bottom and know nothing about the layers above. The service owns all data access. The clients present results.

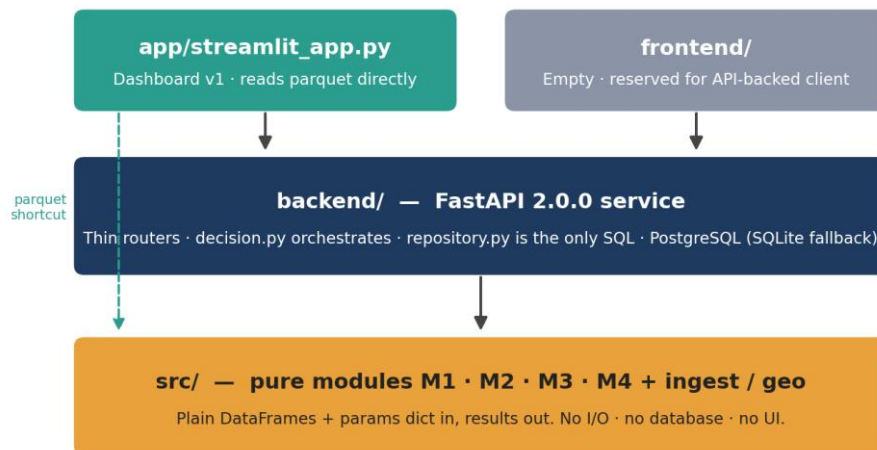


Figure 4.1. Three-layer architecture. The Streamlit dashboard still reads parquet directly; 'frontend/' is reserved.

| Layer        | Location                                      | Responsibility                                                                                                                                   |
|--------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Pure modules | <code>src/</code>                             | M1–M4 plus ingest and geo. No I/O, no database, no UI.                                                                                           |
| Service      | <code>backend/</code>                         | FastAPI 2.0.0 over PostgreSQL (SQLite fallback). Owns all data access. Routers are thin; orchestration is in <code>services/decision.py</code> . |
| Clients      | <code>app/</code> ,<br><code>frontend/</code> | <code>app/streamlit_app.py</code> (856 lines) is the v1 default UI. <code>frontend/</code> is empty.                                             |

### 4.2 The Purity Contract

Every module in `src/` is a pure function over plain DataFrames or arrays plus a parameters dictionary. `backend/services/repository.py` is the only place in the codebase where SQL is written, and it returns DataFrames whose column names are exactly those the modules already expect. The mathematics is therefore identical whether a row came from a parquet file or from PostgreSQL.

| Consequence                              | Why it holds                                |
|------------------------------------------|---------------------------------------------|
| Modules can be tested without a database | They take frames, not connections.          |
| Data access is auditable in one place    | Only <code>repository.py</code> writes SQL. |

| Consequence                    | Why it holds                                             |
|--------------------------------|----------------------------------------------------------|
| The storage backend can change | The column contract stays constant.                      |
| Results are reproducible       | No hidden state, clock or network call inside the maths. |

### 4.3 Two Deliberately Separate Client Paths

**Limitation** The Streamlit dashboard, the default UI, still reads parquet directly instead of calling the API. `frontend/` is empty and reserved for a client that talks to the API. That client is planned work, not a current capability.

This split is deliberate. The dashboard was built first and proves the mathematics; the service was built second and proves that the same mathematics survives a database. Because both call the same pure modules, moving the dashboard onto the API is a wiring task and not a rewrite.



Each module reads the previous stage's output. All are pure functions.

*Figure 4.2. The analytical pipeline. Each module consumes the output of the previous stage.*

## 5. Data Model

### 5.1 Tables

The schema is defined once, in `backend/models.py`, using SQLAlchemy 2.0 and created with `create_all`. There are six tables.

| Table          | Primary key       | Rows   | Holds                                                            |
|----------------|-------------------|--------|------------------------------------------------------------------|
| prices         | id                | 14,352 | The Agmarknet fact table                                         |
| mandis         | market            | 72     | District, latitude, longitude, geocode provenance                |
| farms          | farm_id           | 1      | Default FPO depot <code>fpo_davangere</code> at 14.30, 76.00     |
| farm_distances | (farm_id, market) | 72     | Cached farm-to-mandi distance matrix                             |
| params         | name              | 1      | Assumption set in <code>params.yaml</code> shape, stored as JSON |
| dataset_meta   | key               | 9      | Provenance and load timestamp                                    |

### 5.2 Indices and the Query Shape Behind Them

Every decision query has the same shape: one crop, one mandi, up to a date. The composite index on `(commodity, market, date)` serves that shape directly. The remaining indices support cross-mandi scans and distance lookups.

| Index                      | Columns                   | Purpose                                                 |
|----------------------------|---------------------------|---------------------------------------------------------|
| ix_prices_crop_market_date | (commodity, market, date) | Single-series reads for one crop and mandi up to a date |
| ix_prices_date             | date                      | Date-bounded scans                                      |
| ix_prices_crop_date        | (commodity, date)         | Cross-mandi reads for one crop                          |
| ix_farm_distances_farm     | farm_id                   | Distance lookup for a depot                             |

### 5.3 Repository Behaviours

`services/repository.py` has two behaviours that are easy to miss and important to know.

| Behaviour     | Detail                                                                                                    |
|---------------|-----------------------------------------------------------------------------------------------------------|
| Date coercion | The <code>date</code> column is coerced to <code>datetime64</code> on read, so modules never see strings. |
| Caching       | Reads are cached for the process. Any write invalidates the cache.                                        |

## 5.4 Loading the Database

`backend/seed/load_db.py` is the ETL entry point and is run by `make reset-db`. It populates the tables above, including the provenance and load timestamp held in `dataset_meta`. The complete flag set was not available when this report was written.

## 6. Analytical Modules

The four modules run in pipeline order. Each is a pure function. This chapter states the problem, the method, every formula, and the reason the method was chosen over the obvious alternative.

### 6.1 M1: Econometrics (src/m1\_econometrics/stl\_bands.py)

#### Problem

Describe the seasonal and trend structure of a mandi's price, flag abnormal weeks, and produce a deterministic price path over the decision horizon.

#### Method

The weekly median price is log-transformed and decomposed with a robust STL (seasonal-trend decomposition using LOESS) with a 52-week period. The residual volatility gives  $\pm 2\sigma$  bands. Residual z-scores mark weeks as GLUT (unusually low) or SPIKE (unusually high). The price path is then built from two parts.

$p(d) = \text{trend extension} + \text{seasonal replay}$

trend extension : OLS slope fitted on the last 26 weeks of the trend component

seasonal replay : seasonal component from the same calendar week last year

A conservative toggle shifts the whole path to the lower volatility band, for users who want to plan against a weaker market.

#### Outputs for the Worked Example

| Diagnostic            | Onion at Davangere                  |
|-----------------------|-------------------------------------|
| Weekly points         | 106 (94 observed, 11% interpolated) |
| $F_{\text{seasonal}}$ | 0.9996                              |
| $F_{\text{trend}}$    | 0.9993                              |
| GLUT / SPIKE weeks    | 3 / 0                               |
| Seasonal index range  | 0.62 to 1.56                        |
| Peak and trough       | November and May                    |

The November peak and May trough match the known Karnataka onion cycle, which is a useful sanity check on the decomposition.

#### Why STL and Not Prophet

Prophet fits changepoints, and in doing so hides them: a sudden break becomes part of a smooth curve. STL exposes its trend, seasonal and residual components and the F-

statistics for each, so a reviewer can see when the decomposition is weak. For a committee that must argue with the output, visibility outranks flexibility.

**Assumption** The price path is a seasonal-trend extrapolation. It is not a live quote and not a guarantee of any future price.

## 6.2 M2: Spatial Arbitrage (src/m2\_arbitrage/nhr.py)

### Problem

Rank mandis by the rupees per quintal that actually reach the FPO, not by board price.

### Method: The Cost Walk

```
r_km    = diesel / mileage + other
trucks  = ceil(Q / capacity)
freight = trucks × (fixed + return_factor × r_km × km)
gross   = Q × (1 - shrink) × board
fees    = gross × (cess + commission)
handling = Q × (load + unload) + fixed
net     = gross - fees - handling - freight
```

With default parameters **r\_km** is ₹30.5 per kilometre. Distance is the haversine (great-circle) distance multiplied by a circuitry factor of 1.3, which approximates road distance without a live routing service. Live routing was rejected because it needs the network, has rate limits and produces results that change from run to run.

Only mandis with at least one quote inside the recent window (default 14 days) are eligible. If none qualify, the system shows no SELL AT card and says why.

## 6.3 M3: Hold Versus Sell (src/m3\_milp/)

### Problem

Decide whether holding stock beats selling now, on which day it starts to, and how to split a lot across several sale dates.

### Step A: The Carry Cost

```
c = rent/30 + insurance + P/365 × ( LTV × loan + (1 - LTV) × opportunity )
```

**P** is the price level, **LTV** the loan-to-value ratio, **loan** the loan interest rate and **opportunity** the FPO's opportunity rate on the equity share. At default parameters the carry cost is **₹0.92 per quintal per day**.

### Step B: The Value Curve and Break-Even

$$\theta = 1 - \text{shrink\_per\_day}$$

$$V(t) = \theta^t \times p(t) - c \times t - \text{one\_time\_fee} \times [t > 0]$$

break-even = first  $t$  where  $V(t) \geq V(0)$

$V(t)$  is the value per quintal of selling on day  $t$ . It rises with the projected price and falls with shrinkage and carry cost. Break-even is the first day on which holding is worth at least as much as selling immediately.

### Step C: The Multi-Tranche MILP

A single sale date is rarely optimal, so the schedule is solved as a MILP with PuLP and the CBC solver.

| Element         | Setting                                                    |
|-----------------|------------------------------------------------------------|
| Inventory       | Balance constraint each day; terminal condition $I(T) = 0$ |
| Tranche cap     | $K = 3$ sale dates at most                                 |
| Minimum tranche | 10% of stock, to avoid trivial sales                       |
| Storage charge  | 30-day block charge, modelled with a binary variable       |

Commercial solvers such as Gurobi and CPLEX were rejected on licensing grounds. `scipy` has no MILP solver. CVXPY adds overhead the problem does not need.

**Limitation** Three tranches is an approximation of a continuous selling schedule. It is a deliberate trade of optimality for a schedule a manager can actually execute.

## 6.4 M4: Aggregation (src/m4\_aggregation/knapsack.py)

### Problem

A buyer wants a fixed quantity  $T$ . Farmers offer lots of different sizes. Choose lots that reach  $T$  with the least surplus.

### Method

minimise  $\Sigma - T$   
 subject to  $T \leq \Sigma \leq T \times (1 + \varepsilon)$   
 per-lot multiplicity respected  
 no single farmer above 40% of the order

An independent 0/1 subset-sum dynamic programme with complexity  $O(N \cdot S)$  recomputes the minimum surplus. If the two agree, the result is confirmed by two unrelated methods. If the pool cannot reach the order, the solver returns a genuine **Infeasible** status and never a fabricated zero-quantity fill.

## 7. API Service

### 7.1 Operations

The service exposes eleven operations across ten paths under the base `/api`. Every operation is a `GET` except one `PUT`. Routers are thin; orchestration lives in `services/decision.py`.

| Operation                                    | Purpose                              |
|----------------------------------------------|--------------------------------------|
| <code>GET /api/health</code>                 | Liveness check                       |
| <code>GET /api/meta</code>                   | Dataset provenance and geocode split |
| <code>GET /api/decision</code>               | All three answers in one response    |
| <code>GET /api/m1 ... /api/m4</code>         | Individual module outputs            |
| <code>GET /api/eligible-mandis</code>        | Mandis quoting inside the window     |
| <code>GET /api/params</code>                 | Read the assumption set              |
| <code>PUT /api/params</code>                 | Replace the assumption set           |
| <code>GET /api/export/clean-slice.csv</code> | Export the cleaned data slice        |

### 7.2 Shared Query String

`crop` and `as_of` are required on decision routes. Every other parameter has a default and a validated range.

| Parameter                       | Default  | Bounds                         |
|---------------------------------|----------|--------------------------------|
| <code>variety</code>            | none     | Valid varieties for the crop   |
| <code>window_days</code>        | 14       | 1–60                           |
| <code>volume</code>             | 200 qtl  | 10–1000                        |
| <code>horizon</code>            | 180 days | 30–240                         |
| <code>conservative</code>       | off      | Boolean                        |
| <code>farm_lat, farm_lon</code> | depot    | Coordinates                    |
| <code>n_farmers</code>          | 25       | 5–60                           |
| <code>order</code>              | 400 qtl  | 50–1000                        |
| <code>ref_mandi</code>          | none     | Valid mandi                    |
| <code>overrides</code>          | none     | JSON of dotted parameter paths |

### 7.3 Error Contract

| Code | Meaning                                                                                                   |
|------|-----------------------------------------------------------------------------------------------------------|
| 400  | Unknown crop, variety, <b>as_of</b> , or malformed <b>overrides</b> . The message names the valid values. |
| 422  | A numeric parameter is outside its bounds.                                                                |
| 503  | The database is empty or unreachable.                                                                     |

Validation is declared once, next to the response shape, using Pydantic. This is why FastAPI was chosen: the contract and the documentation cannot drift apart.

### 7.4 Caching

| Cache                 | Detail                                            |
|-----------------------|---------------------------------------------------|
| <b>build_decision</b> | 64-entry FIFO                                     |
| Frame cache           | Held for the process                              |
| Invalidation          | <b>invalidate_caches</b> is called on every write |

## 8. User Interface

The Streamlit dashboard ([app/streamlit\\_app.py](#), 856 lines) presents the three answers as cards above the supporting evidence. A sidebar acts as the control room, exposing crop, decision date, volume, horizon, order size and the conservative toggle. Charts use Plotly and the map uses folium through streamlit-folium.

### 8.1 Motion That Carries Meaning

Animation in the dashboard is deliberate. Each animated element explains something about the computation, and none is decoration.

| Element               | What it communicates                        |
|-----------------------|---------------------------------------------|
| Pipeline dots         | Which module is currently computing         |
| Bar-rank reveal       | The ordering of mandis by net realisation   |
| Projection sweep      | The price path unfolding across the horizon |
| Pulsing origin marker | The location of the FPO depot on the map    |

### 8.2 Marking Approximation

Mandis located by district centroid are visibly marked on the map. A stale-data warning stays on screen, because a 475-day-old dataset should never look current.

**Limitation** A numbered top-to-bottom walkthrough of the dashboard journey was not available when this report was written. See [REPORT.md](#) for the sequence.

## 9. Worked Example

This chapter reproduces the default run. It is the section that demonstrates the system working end to end.

|                            |                  |
|----------------------------|------------------|
| <b>Crop</b>                | Onion            |
| <b>Decision date</b>       | 2025-06-11       |
| <b>Volume</b>              | 200 quintals     |
| <b>Depot</b>               | 14.30 N, 76.00 E |
| <b>Recent-quote window</b> | 14 days          |
| <b>Horizon</b>             | 180 days         |
| <b>Order</b>               | 400 quintals     |

### 9.1 The Three Answers

| Card       | Result                                                                       |
|------------|------------------------------------------------------------------------------|
| SELL AT    | Channarayapatna, net <b>₹2,756 per quintal</b> , 211 km                      |
| HOLD       | <b>157 days</b> , break-even on day 4, gain <b>₹2,182 per quintal</b>        |
| BULK ORDER | 400 quintals, surplus 0, 8 farmers from a pool of 25 offering 1,373 quintals |

### 9.2 SELL AT: The Cost Walk

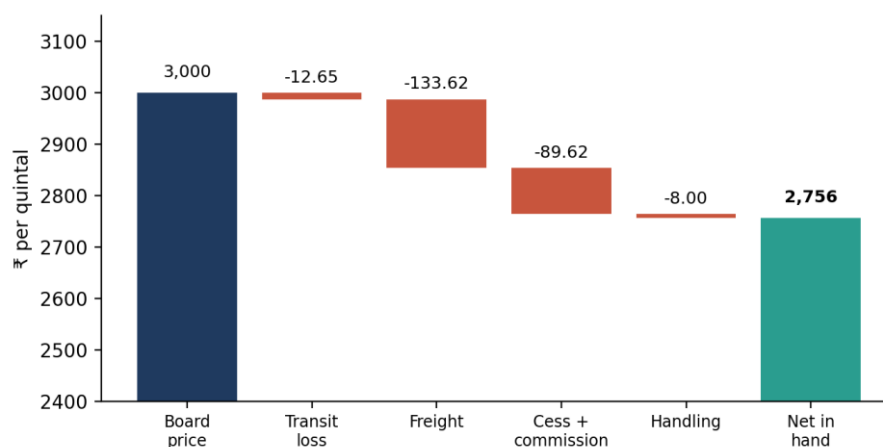


Figure 9.1. Cost walk for Channarayapatna, per quintal (vertical axis starts at ₹2,400).

| Line         | ₹ per quintal |
|--------------|---------------|
| Board price  | 3,000.00      |
| Transit loss | -12.65        |
| Freight      | -133.62       |

| Line                | ₹ per quintal |
|---------------------|---------------|
| Cess and commission | -89.62        |
| Handling            | -8.00         |
| <b>Net in hand</b>  | <b>2,756</b>  |

The lines reconcile:  $3,000 - 12.65 - 133.62 - 89.62 - 8.00 = 2,756.11$ .

| Robustness check                 | Result                                                          |
|----------------------------------|-----------------------------------------------------------------|
| Nearest mandi, Davangere (25 km) | Board ₹1,600, net ₹1,523                                        |
| Diesel raised to ₹130 per litre  | Winner still nets ₹2,714. The choice is not knife-edge on fuel. |
| Mandis quoting inside the window | 12 of 43 onion mandis                                           |

### 9.3 M1 Diagnostics

The reference series is Davangere. It has 106 weekly points, of which 94 are observed and 11% are interpolated. The decomposition is strong:  $F_{\text{seasonal}}$  is 0.9996 and  $F_{\text{trend}}$  is 0.9993. The series shows 3 GLUT weeks and no SPIKE weeks. The seasonal index runs from 0.62 to 1.56, peaking in November and bottoming in May, which is the correct Karnataka onion cycle.

### 9.4 HOLD: Value Curve and MILP

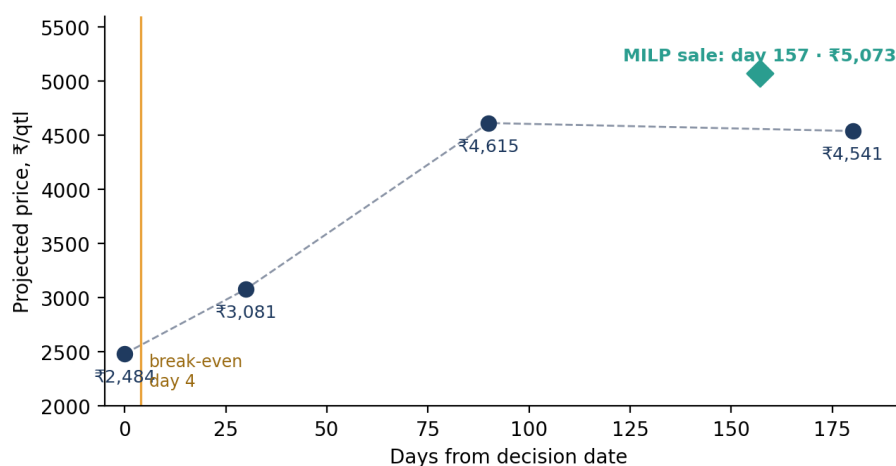


Figure 9.2. Projected price markers, break-even and the MILP sale day.

| Marker  | Projected price (₹ per quintal) |
|---------|---------------------------------|
| Day 0   | 2,484                           |
| Day 30  | 3,081                           |
| Day 90  | 4,615                           |
| Day 180 | 4,541                           |

| MILP result           | Value                                     |
|-----------------------|-------------------------------------------|
| Schedule              | One tranche of 189.8 quintals on day 157  |
| Sale price            | ₹5,073                                    |
| Objective             | ₹931,330                                  |
| Why 189.8 and not 200 | Storage shrinkage consumed the difference |

The break-even curve and the MILP agree on day 157 as the sale day. Two independent formulations reaching the same answer is the strongest evidence in the report that the hold logic is sound.

### 9.5 BULK ORDER: Knapsack

From a pool of 25 farmers offering 1,373 quintals, the knapsack selects 8 farmers to fill exactly 400 quintals, leaving zero surplus. The independent dynamic programme returns the same zero surplus.

**Note** The mandis, distances and prices in this example are real. The farmer lot sizes are simulated with seed 11, so the BULK ORDER result demonstrates the method, not a real farmer roster.

## 10. Testing and Quality Assurance

### 10.1 Test Layers

| Layer              | Tooling                                   | File (lines)                        |
|--------------------|-------------------------------------------|-------------------------------------|
| Module mathematics | pytest                                    | <code>test_modules.py</code> (43)   |
| Data pipeline      | pytest                                    | <code>test_pipeline.py</code> (158) |
| Dashboard          | <code>streamlit.testing.v1.AppTest</code> | <code>test_app.py</code> (113)      |
| API                | <code>fastapi.testclient</code>           | Not yet committed                   |

The documented total is 43 tests, split 6, 19 and 18 across the three committed files. That figure is quoted from project documentation; the test suite was not executed when this report was written.

### 10.2 What Each Layer Catches

Mathematics tests catch wrong formulas. They cannot catch a dashboard that feeds the right formula the wrong data. That is why the dashboard is tested through `AppTest`, which drives the real application. `test_app.py` caught two bugs that no maths test could have found; one is the crop-mixing defect described in Section 3.4.5.

### 10.3 API Verification

Verifying the API against the live service surfaced four bugs. All four were the same species: a Python type leaking across a serialisation boundary. A value that behaves correctly inside Python can fail or change meaning once it crosses into JSON or a database driver.

**Limitation** The API verification harness has not yet been committed to `tests/`. Until it is, those four fixes are not protected by a regression test.

### 10.4 Reproducibility

The pipeline is deterministic end to end. Geocoding is cached to CSV, distances are computed and stored, lot sizes use a fixed seed, and every decision takes an explicit `as_of` date. Running `make data`, `make reset-db` and `make test` on a clean checkout reproduces the numbers in Chapter 9.

## 11. Design Rationale: Why There Is No Machine Learning

This is a design argument, not a religion. A cooperative deciding whether to hold 100 quintals of onion needs to know why the system recommends it, and needs to be able to argue with the recommendation. A fitted forecast hides its error bars behind a single point estimate and hides its changepoints behind a smooth line. In this system, the STL components, F-statistics, residual z-scores, cost terms, MILP schedule and knapsack selections are all inspectable.

| Property          | This system                          | A fitted forecaster            |
|-------------------|--------------------------------------|--------------------------------|
| Uncertainty       | Explicit $\pm 2\sigma$ bands         | Hidden behind a point estimate |
| Structural breaks | Visible in STL residuals             | Absorbed into a smooth curve   |
| Cost terms        | Each line can be read and challenged | Not applicable                 |
| Hold schedule     | MILP with visible tranches and caps  | Opaque                         |
| Reproducibility   | Identical output on identical input  | Depends on the training run    |

The claim is deliberately narrow. Machine learning may suit other problems. For this one, inspectability is worth more than a marginal gain in accuracy that could not be verified on a dataset this thin.

The system therefore produces a deterministic seasonal-trend projection. It does not claim to predict, and the report avoids stronger words than the method deserves.

## 12. Limitations and Integrity

### 12.1 Limitations

Each limitation below is specific and, where possible, quantified. They are design facts to be stated plainly, not defects to apologise for.

| #  | Limitation                  | Quantification                                                                               |
|----|-----------------------------|----------------------------------------------------------------------------------------------|
| 1  | Stale data                  | Ends 11 June 2025; 475 days old at 29 September 2026                                         |
| 2  | Projection is extrapolation | Not a live quote and not a guarantee                                                         |
| 3  | Approximate distances       | Haversine $\times$ 1.3, not road distance                                                    |
| 4  | Location precision          | 13 of 72 mandis pinned to district centroids                                                 |
| 5  | Simulated lot sizes         | Seed 11. Mandis, distances and prices are real                                               |
| 6  | No arrivals column          | GLUT and SPIKE are price-based proxies                                                       |
| 7  | Unverified cost rates       | Every <code>params.yaml</code> rate is an assumption until its <code>source</code> is filled |
| 8  | MILP approximation          | Three tranches at most                                                                       |
| 9  | Crop coverage               | Only Onion and Potato support the annual model                                               |
| 10 | One-sided unit guard        | Could mask a genuine crash of more than 90%                                                  |
| 11 | API maturity                | No committed API tests, no migrations, no authentication                                     |

### 12.2 The System Refuses to Fabricate

A decision tool that always returns an answer is dangerous. AgriLink-OR is built to return a reason instead when the data cannot support an answer.

| Situation                          | Behaviour                                                                                  |
|------------------------------------|--------------------------------------------------------------------------------------------|
| Thin crop (Tomato, Wheat, Rice)    | <code>m1.error</code> is returned and <code>m3.available</code> is false. No fit to noise. |
| No recent quotes                   | No SELL AT card is shown.                                                                  |
| Pool cannot reach the order        | A real <code>Infeasible</code> status is returned, not a zero-quantity fill.               |
| Unknown crop or <code>as_of</code> | A named <code>400</code> lists the valid values.                                           |
| Old data                           | The stale-data warning stays visible.                                                      |

## 13. Roadmap

The following work is proposed in priority order. Items 1 to 3 close the gaps that most affect trust in the current results.

| # | Item                                                                                | Closes              |
|---|-------------------------------------------------------------------------------------|---------------------|
| 1 | Commit the API test harness to <code>tests/</code>                                  | Section 10.3        |
| 2 | Fill the <code>source:</code> field for every <code>params.yaml</code> rate         | Limitation 7        |
| 3 | Build the API-backed client in <code>frontend/</code>                               | Section 4.3         |
| 4 | Refresh the data with <code>make fetch</code> (requires <code>DATA_GOV_KEY</code> ) | Limitation 1        |
| 5 | Add database migrations and authentication                                          | Limitation 11       |
| 6 | Replace simulated lots with real FPO lot data                                       | Limitation 5        |
| 7 | Move from haversine to road distances and resolve the 13 fallback mandis            | Limitations 3 and 4 |
| 8 | Replace the one-sided unit guard with a per-commodity two-sided rule                | Limitation 10       |

## Appendix A. Commands and Runtime

| Command                    | Purpose                                                            |
|----------------------------|--------------------------------------------------------------------|
| <code>make setup</code>    | Install dependencies                                               |
| <code>make data</code>     | Build the data pipeline (about 3 minutes)                          |
| <code>make reset-db</code> | Rebuild the database                                               |
| <code>make test</code>     | Run the tests (43 documented)                                      |
| <code>make app</code>      | Start the dashboard on port 8501                                   |
| <code>make backend</code>  | Start the API on port 8000; interactive docs at <code>/docs</code> |
| <code>make fetch</code>    | Fetch fresh data (requires <code>DATA_GOV_KEY</code> )             |

## Appendix B. Technology Choices

| Layer          | Choice                                                         | Rejected alternative and reason                                       |
|----------------|----------------------------------------------------------------|-----------------------------------------------------------------------|
| Frames         | pandas, pyarrow                                                | Parquet is about 5× smaller and preserves dtypes                      |
| Database       | PostgreSQL via SQLAlchemy 2.0                                  | SQLite kept as a zero-setup fallback                                  |
| API            | FastAPI, Uvicorn                                               | Pydantic validation is declared once, beside the response shape       |
| Decomposition  | statsmodels <code>STL</code>                                   | Prophet fits changepoints and hides them                              |
| Optimisation   | PuLP 2.9.0 with CBC                                            | Gurobi/CPLEX (licence), scipy (no MILP), CVXPY (overhead)             |
| Distances      | haversine × 1.3                                                | Live routing needs a network, has rate limits and is not reproducible |
| Interface      | Streamlit                                                      | Dash and Flask add nothing needed here                                |
| Charts and map | Plotly, folium, streamlit-folium                               | None rejected                                                         |
| Geocoding      | requests to Nominatim, cached to CSV                           | <code>geopy</code> is not needed                                      |
| Configuration  | PyYAML <code>params.yaml</code>                                | Every rate carries a <code>source:</code> field                       |
| Tests          | pytest, <code>AppTest</code> , <code>fastapi.testclient</code> | None rejected                                                         |

## Appendix C. Assumption Parameters

Every rate in `data/ref/params.yaml` is an assumption until its `source:` field is filled. Values and source status were not available when this report was written, so only parameter groups are listed.

| Group       | Parameters                                                                                 |
|-------------|--------------------------------------------------------------------------------------------|
| Freight     | Diesel price, mileage, other per-kilometre cost, truck capacity, fixed cost, return factor |
| Fees        | Cess, commission                                                                           |
| Handling    | Load, unload, fixed charge                                                                 |
| Loss        | Transit shrinkage, per-day storage shrinkage                                               |
| Storage     | Rent, insurance, one-time fee                                                              |
| Finance     | Loan rate, loan-to-value, opportunity rate                                                 |
| Solver      | Tranche cap <b>K</b> (3), minimum tranche (10%), block charge (30 days)                    |
| Aggregation | Surplus tolerance <b>ε</b> , concentration cap (40%)                                       |

**Assumption** None of these rates should be presented as a measured fact. They are working assumptions and can be overridden per request through the `overrides` query parameter.

## Appendix D. Repository Map

| Path                                            | Responsibility                           |
|-------------------------------------------------|------------------------------------------|
| <code>src/m1_econometrics/stl_bands.py</code>   | STL, volatility bands, flags, price path |
| <code>src/m2_arbitrage/nihp.py</code>           | Net in-hand realisation                  |
| <code>src/m3_milp/</code>                       | Carry cost, break-even, hold/sell MILP   |
| <code>src/m4_aggregation/knapsack.py</code>     | Knapsack and DP cross-check              |
| <code>src/data/load_history.py, clean.py</code> | Date parsing and the unit-error guard    |
| <code>src/geo/</code>                           | Geocoding and distance matrix            |
| <code>backend/models.py, schemas.py</code>      | Database schema and API shapes           |
| <code>backend/services/repository.py</code>     | The only SQL surface                     |
| <code>backend/services/decision.py</code>       | Orchestration of the three answers       |
| <code>backend/routers/</code>                   | Thin route handlers                      |
| <code>backend/seed/load_db.py</code>            | ETL into the database                    |
| <code>app/streamlit_app.py</code>               | Dashboard (856 lines)                    |

| Path                 | Responsibility                                 |
|----------------------|------------------------------------------------|
| tests/               | test_modules.py, test_pipeline.py, test_app.py |
| data/ref/params.yaml | The assumption set                             |

## Appendix E. Glossary

| Term          | Meaning                                                                                           |
|---------------|---------------------------------------------------------------------------------------------------|
| Mandi         | A wholesale agricultural market                                                                   |
| Quintal (qtl) | 100 kilograms                                                                                     |
| Modal price   | The most common price quoted in a market on a given day                                           |
| FPO           | Farmer Producer Organisation                                                                      |
| Cess          | A market fee charged as a percentage of sale value                                                |
| STL           | Seasonal-trend decomposition using LOESS; splits a series into trend, seasonal and residual parts |
| MILP          | Mixed-integer linear programme; an optimisation with some whole-number decisions                  |
| Break-even    | The first day on which holding is worth at least as much as selling now                           |
| Knapsack      | Choosing items to reach a target size with minimum waste                                          |
| GLUT / SPIKE  | Weeks where price sits unusually far below or above the seasonal-trend fit                        |
| Haversine     | Great-circle distance between two latitude/longitude points                                       |

## Appendix F. Verification Notes

This report was written from the project's ground-truth summary, which distils [README.md](#), [REPORT.md](#), [explain.md](#), [Makefile](#), [requirements.txt](#) and [params.yaml](#). The repository itself was not available to the writer. The table lists each discrepancy and how it was resolved, and each item that could not be verified.

| Item                                                                        | Resolution                                                                                     |
|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Onion rows: 6,349 in <a href="#">params.yaml:4</a> , 6,330 in the documents | Used 6,330, the value that makes the five crops sum to 14,352. The figure 6,349 is unverified. |
| "Four traps" versus five enumerated                                         | Used five. The ground truth enumerates five.                                                   |
| Exact library versions in prose                                             | Not stated as facts. Only PuLP 2.9.0 was given as pinned.                                      |

| Item                                                                                                        | Resolution                                                                                        |
|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| 59 resolved versus 72 total                                                                                 | 59 + 13 = 72, reported as 59 <b>market</b> and 13 <b>district_fallback</b> .                      |
| Staleness of about 475 days                                                                                 | Recomputed: 11 June 2025 to 29 September 2026 is 475 days.                                        |
| Test count of 43                                                                                            | Quoted from documentation. Not re-run.                                                            |
| Per-crop median and range                                                                                   | Not available. Omitted.                                                                           |
| Names of the tests locking each trap                                                                        | Not available. Marked in Section 3.4.                                                             |
| Response keys, exact error text, ETL flags, dashboard walkthrough, second <b>AppTest</b> bug, four API bugs | Not available. Omitted or marked in place.                                                        |
| Line-level citations                                                                                        | Only <b>load_history.py:44</b> and <b>params.yaml:4</b> were supplied. Others cite the file only. |